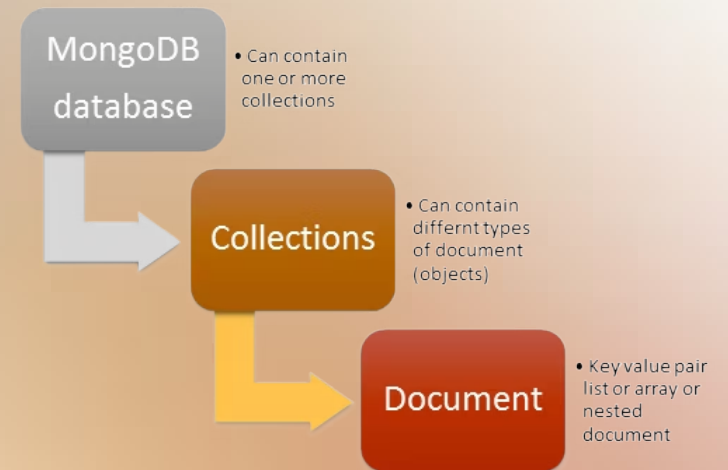
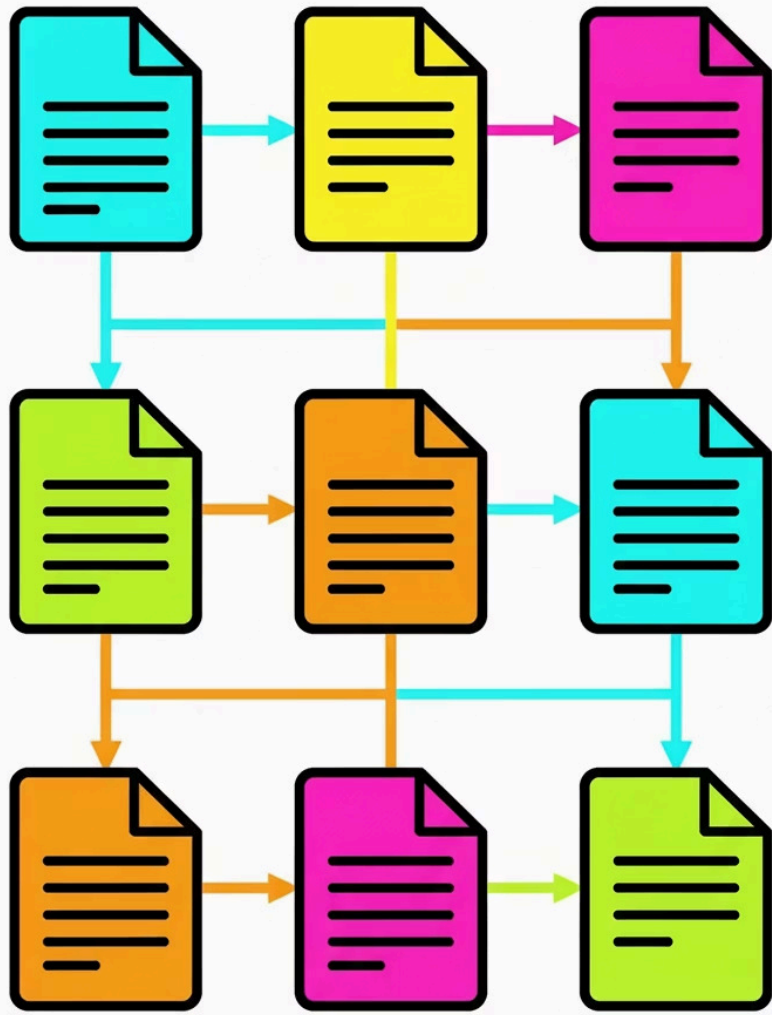


Estrutura de Coleções de Documentos em MongoDB





Anatomia das Coleções MongoDB

Múltiplas Coleções

Organize dados em grupos lógicos e independentes

Tipos Diversos

Diferentes estruturas de documentos na mesma coleção

Pares Chave-Valor

Arrays, listas e documentos aninhados

Flexibilidade de Esquema: Schemaless?

```
{  
  "name": "Juca Pirama",  
  "title": "Director of Developer Advocacy",  
  "Address": {  
    "address1": "Latin Hall",  
    "address2": "Golden Lane",  
    "eircode": "D09 N623"  
  },  
  "expertise": ["MongoDB", "Python", "Javascript"],  
  "employee_number": 320,  
  "location": [53.34, -6.26]  
}
```

Validação de Esquema no MongoDB

Embora o MongoDB seja flexível por natureza, a validação de esquema permite impor regras de estrutura e tipo de dados em coleções, garantindo a consistência e integridade dos documentos.

Consistência de Dados

Define e garante a estrutura esperada para os documentos.

Qualidade e Integridade

Evita a inserção de dados inconsistentes ou incorretos.

Desenvolvimento Facilitado

Reduz erros e agiliza a criação de aplicações.

Controle de Tipos

Impede a inserção de tipos de dados não autorizados.

Exemplo de Validação de Esquema

Para garantir a integridade dos dados, podemos definir regras específicas para a estrutura e o tipo dos campos em uma coleção MongoDB.

```
const userSchema = {  
  bsonType: 'object',  
  required: ['name', 'isAdmin'],  
  properties: {  
    name: {  
      bsonType: 'string',  
      minLength: 5,  
      description: 'must be a string and is required'  
    },  
    isAdmin: {  
      bsonType: 'bool',  
      description: 'must be a boolean and is required'  
    }  
  }  
};
```

1

Tipo do Documento

`bsonType: 'object'` assegura que o documento raiz é um objeto.

2

Campos Obrigatórios

`required: ['name', 'isAdmin']` força a presença destes campos em cada documento.

3

Regras para 'name'

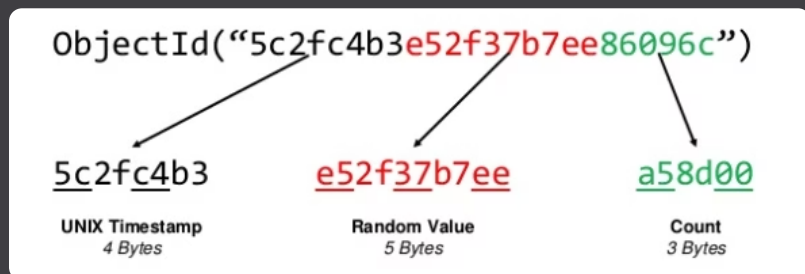
`name` deve ser uma `string` com no mínimo 5 caracteres.

4

Regras para 'isAdmin'

`isAdmin` deve ser do tipo `boolean`.

ObjectId: Identificador Único



Geração Automática

Cada documento recebe um _id único ao ser inserido

- 12 bytes hexadecimais
- Timestamp + identificador único
- Distribuído e sem colisões

Operações CRUD no MongoDB



Create

- `db.collection.insert()`
- `db.collection.save()`
- `db.collection.update({}, {}, {upsert: true})`



Read

- `db.collection.find()`
- `db.collection.findOne()`



Update

- `db.collection.update()`



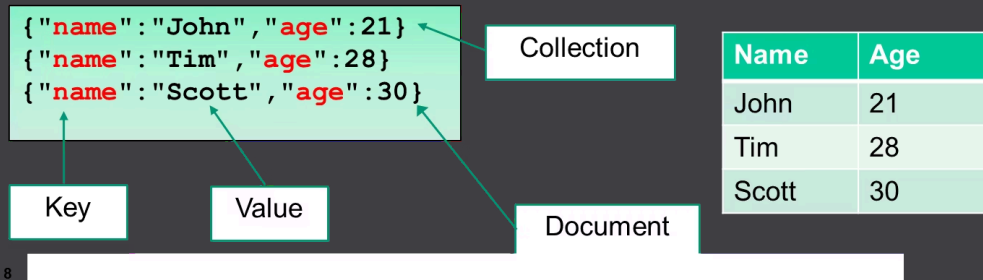
Delete

- `db.collection.remove()`

MongoDB vs SQL: Conceitos Fundamentais

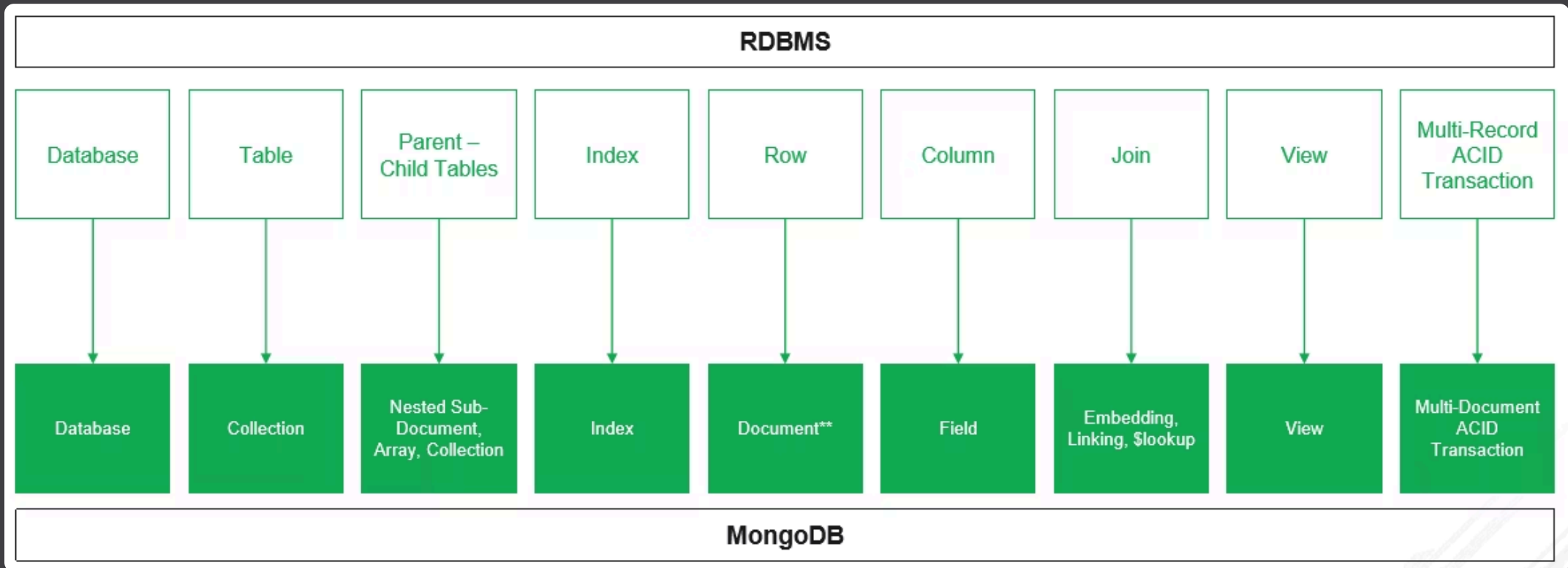
Terminologia Relacional

Mongo/NoSQL Terms	Traditional SQL Terms
FIND	SELECT
SAVE	INSERT
REMOVE	DELETE
UPDATE	UPDATE
ensureIndex	CREATE INDEX
SORT()	ORDER BY
LIMIT	LIMIT/FIRSTN



Mapeamento direto entre conceitos SQL e MongoDB facilita a transição

- Database → Database
- Table → Collection
- Row → Document
- Column → Field



Índices ESR: Maximizando o Desempenho de Queries

A estratégia ESR (Equality, Sort, Range) é fundamental para construir índices que otimizam o tempo de resposta das suas consultas no MongoDB, garantindo que os campos mais seletivos e as operações de ordenação e intervalo sejam considerados de forma eficiente.

```
/**
 * Create the index to support the following query:
 * issueDetails.find({
 *   'user._id': userID,
 *   borrowDate: { $gte: date },
 * }, {
 *   sort: { returnedDate: -1 }
 * })
 */
await collections?.issueDetails?.createIndex({
  // Equality
  'user._id': 1,
  // Sort
  returnedDate: 1,
  // Range
  borrowDate: 1,
});
```

1

Campo de Igualdade

'user._id': 1 otimiza a busca exata por um ID de usuário específico.

2

Campo de Ordenação

returnedDate: 1 permite ordenar eficientemente os resultados por data de devolução.

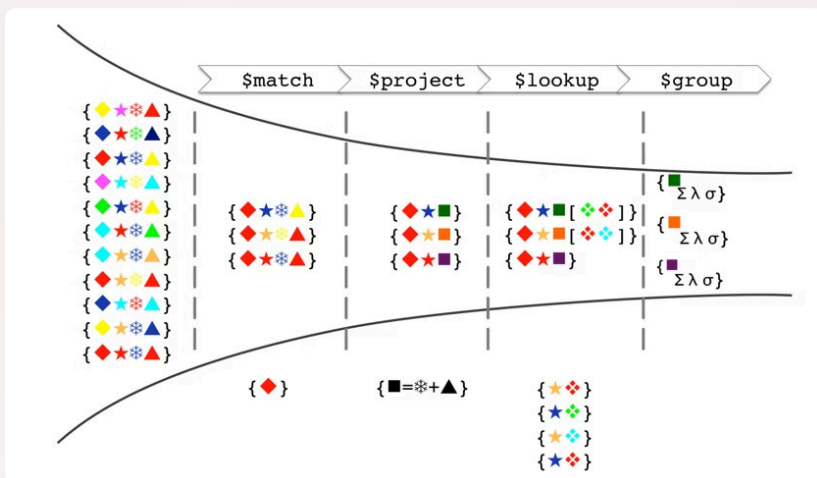
3

Campo de Intervalo

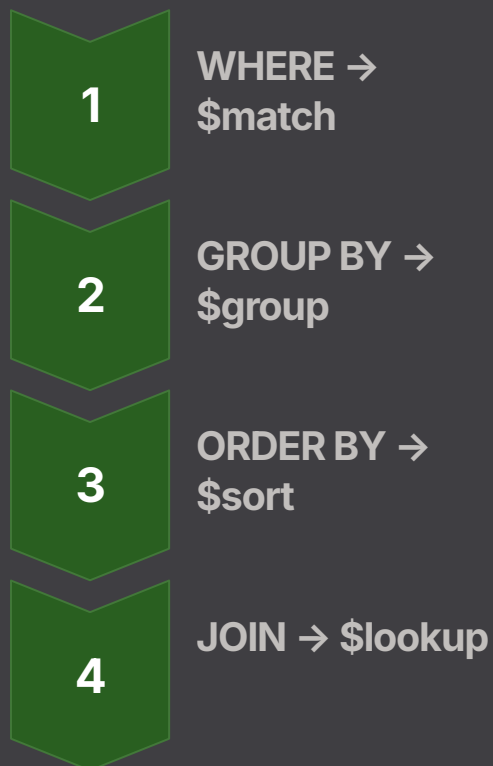
borrowDate: 1 acelera as consultas que buscam documentos dentro de um intervalo de datas.

Aggregation Framework

Pipeline de Transformação



SQL → MongoDB



Pipeline em Etapas

Transforme dados através de múltiplos estágios sequenciais

- Filtragem com \$match
- Agrupamento com \$group
- Projeção com \$project
- Ordenação com \$sort

Queries: Sintaxe Comparada

```
SELECT * FROM usuarios
```

```
> db.usuarios.find()
```

```
SELECT nome FROM usuarios
```

```
> db.usuarios.find({}, {"nome": 1})
```

```
SELECT * FROM usuarios WHERE idade = 29
```

```
> db.usuarios.find({"idade": 29})
```

```
SELECT * FROM usuarios WHERE idade = 29 AND ativo = true
```

```
> db.usuarios.find({"idade": 29, "ativo": true})
```

```
SELECT * FROM usuarios WHERE idade >= 18 AND idade <= 30
```

```
> db.usuarios.find({"idade": {"$gte": 18, "$lte": 30}})
```

```
SELECT * FROM usuarios WHERE nome LIKE "%admin%"
```

```
> db.usuarios.find({"nome": /admin/i})
```

Consultas Equivalentes

Mesmo poder expressivo com sintaxe orientada a documentos JSON

📄 MongoDB usa objetos JSON para filtros, projeções e atualizações

Queries: Sintaxe Comparada

```
SELECT * FROM usuarios ORDER BY nome
> db.usuarios.find().sort({"nome": 1})

SELECT * FROM usuarios ORDER BY idade DESC, nome
> db.usuarios.find().sort({"idade": -1, "nome": 1})

SELECT * FROM usuarios LIMIT 3
> db.usuarios.find().limit(3)

SELECT * FROM usuarios OFFSET 5
> db.usuarios.find().skip(5)

SELECT * FROM usuarios LIMIT 3 OFFSET 5
> db.usuarios.find().limit(3).skip(5)

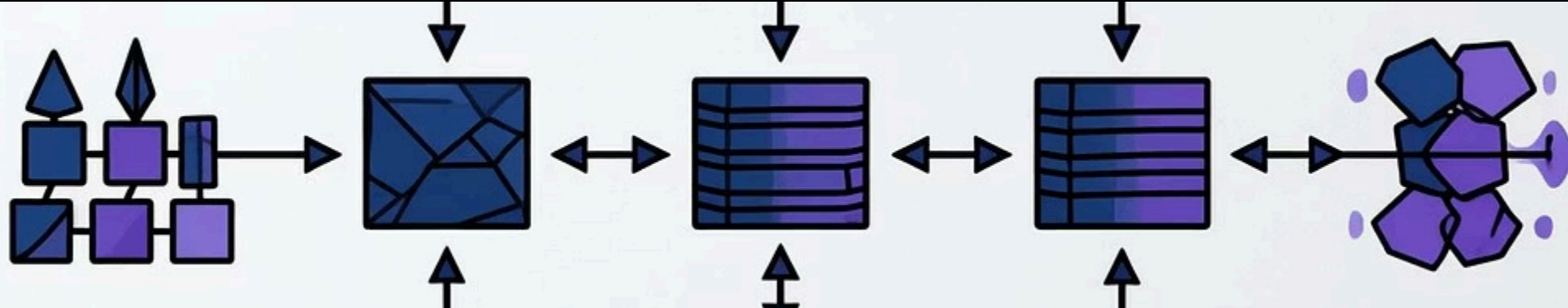
SELECT * FROM usuarios ORDER BY nome LIMIT 3
> db.usuarios.find().sort({"nome": 1}).limit(3)
```

```
INSERT INTO usuarios (nome, idade, ativo)
VALUES ('xunda', 32, true)
> db.usuarios.insert(
  {nome: "xunda", idade: 32, ativo: true})

UPDATE usuarios
SET idade = 28
WHERE nome = 'xunda'
> db.usuarios.update({nome: "xunda"},
  {nome: "xunda", idade: 28, ativo: true})

DELETE FROM usuarios
WHERE nome = "xunda"
> db.usuarios.remove({nome: "xunda"})

DELETE FROM usuarios
> db.usuarios.remove()
```



Exemplos Práticos de Aggregation

LEFT JOIN com \$lookup

```
db.Coleção.aggregate([
  {
    $lookup: {
      from: "outraColeção",
      localField: "campoLocal",
      foreignField: "campoEstrangeiro",
      as: "resultado"
    }
  }
])
```

Agrupamento e Ordenação

```
db.listingsAndReviews.aggregate([
  {
    $group: {
      _id: "$beds",
      min_by_beds: { $min: "$price" }
    }
  },
  { $sort: { min_by_beds: 1 } }
])
```

Filtro Agregado

```
db.zipcodes.aggregate([
  {$group: {
    _id: "$state",
    totalPop: { $sum: "$pop" }
  }},
  {$match: {
    totalPop: { $gte: 10000000 }
  }}
])
```